Level 1 · about 10 h · Python 3.8+

Project: A tip and tax calculator
Examples run on CPython 3.12.3. **You will be able to:**

- Name the basic types int, float, str and bool and check one with type()
- Bind a value to a name and explain what reassignment does
- Use f-strings to build readable text
- Convert between str and numbers and handle the common error

Values, Types and Names

Values have types

Every value has a type. `type()` tells you which.

```
print(type(42))
print(type(3.5))
print(type("hi"))
print(type(True))
```

```
Output (CPython 3.12.3):
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Names point at values

A name is a label stuck to a value. Reassigning moves the label; it does not change the old value.

```
total = 10
total = total + 5
print(total)
```

```
Output (CPython 3.12.3):
15
```

Numbers and arithmetic

/ always gives a float. // divides and rounds down. % gives the remainder.

```
print(7 / 2)
print(7 // 2)
print(7 % 2)
print(2 ** 10)
```

```
Output (CPython 3.12.3):
3.5
3
1
1024
```

Strings and f-strings

An f-string puts values into text. Format specs control decimals.

```
name = "Ada"
price = 19.5
print(f"{name} pays {price:.2f}")
print(name.upper(), len(name))
```

```
Output (CPython 3.12.3):
Ada pays 19.50
ADA 3
```

Converting types

`input()` always returns text. Convert it before doing arithmetic.

```
text = "12"
print(int(text) + 1)
try:
    int("twelve")
```

```
except ValueError as err:
    print("ValueError:", err)
```

```
Output (CPython 3.12.3):
13
ValueError: invalid literal for int() with base 10: 'twelve'
```

Check yourself

1. What is `type(7 / 2)`?
2. After `total = 10` and `total = total + 5`, what does `total` name?
3. Why must you convert the result of `input()` before adding numbers?

Project: Tip and tax calculator

Ask for a bill, a tip percent and a tax percent, convert them, and print the total with two decimals using an f-string.

Sources

- Python tutorial: An informal introduction
- Python reference: The data model
- Python docs: Formatted string literals
- PEP 8 Style Guide for Python Code

Code © CypherMonkeyLabs, MIT. Text CC BY-NC-SA 4.0. Educational material, not accredited coursework; no outcome or income promised. Built 2026-10-18.